

About

Owner @ Tiger Lily Plants 🌿

Activity

Emma Doyle

- You

Owner @ Tiger Lily Plants

7h

Tired of choosing between LinkedIn's awkward PDF export and another resume you have to maintain by hand?

I'm building resumeme, which turns your profile into a professional, readable PDF with proper typography, clickable links, project tiles, and a skills cloud weighted by endorsements: <https://lnkd.in/evwPnqv9>

Just fork the project, sign in to LinkedIn normally, and push the collected data to a project pre-configured with relevant secrets.

No archaic languages or headaches.

Just YAML.

Choose which jobs and projects appear, adjust the theme, or generate summaries tailored to specific employers. There's even a codex integration at-current.

Monthly automated refreshes / resume rebuilds, a personal resume website, and signed releases are available when you want them to prove authenticity.

The coffee stains are automated, too;)

GitHub - astrivant/resumeme: Turn your LinkedIn profile into a polished PDF resume, built and signed with GitHub Actions

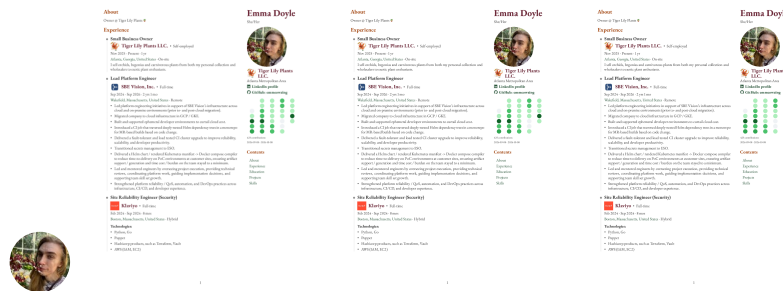
github.com

Comment

Send

79 impressions

View analyticsView analytics



<https://www.linkedin.com/in/emmeowzing/>

Tired of choosing between LinkedIn's awkward PDF export and another resume you have to maintain by hand? I'm building resumeme, which turns your profile into a professional, readable PDF with proper typography, clickable links, project tiles, and a skills cloud weighted by endorsements: <https://lnkd.in/evwPnqv9> Just fork the project, sign in to LinkedIn normally, and push the collected data to a project pre-configured with relevant secrets. No archaic languages or headaches. Just YAML. Choose which jobs and projects appear, adjust the theme, or generate summaries tailored to specific employers. There's even a codex integration at-current. Monthly automated refreshes / resume rebuilds, a personal resume website, and signed releases are available when you want them to prove authenticity. The coffee stains are automated, too;)

GitHub - astrivant/resumeme: Turn your LinkedIn profile into a polished PDF resume, built and signed with GitHub Actions

GitHub - astrivant/resumeme: Turn your LinkedIn profile into a polished PDF resume, built and signed with GitHub Actions
GitHub - astrivant/resumeme: Turn your LinkedIn profile into a polished PDF resume, built and signed with GitHub Actions
github.com

Emma Doyle

She/Her



 **Tiger Lily Plants
LLC.**

Atlanta Metropolitan Area

 **LinkedIn profile**

 **GitHub: emmeowzing**



430 contributions

2026-09-08 - 2026-10-08

Contents

About

Activity

Experience

Education

Projects

Skills

Send

79 impressions View analytics View analytics

Emma Doyle reposted this

Arun Fiddler

• 3rd+

AWS Certified Solution Architect | Sr. DevOps Engineer | Ex-Amazonian | DevOps, AWS Cloud & System Design | Infrastructure Automation | Mentor

Yesterday

👉 Kubernetes 1.37 quietly introduced something DevOps teams are going to love.

In large Kubernetes clusters, it's common for users to create PersistentVolumeClaims (PVCs), use them with Pods, and later delete those Pods without cleaning up the storage.

And Kubernetes doesn't automatically delete the PVC when its associated Pod is removed - intentionally, to protect against accidental data loss.

Over time, these unused PVCs can accumulate, consuming storage capacity and increasing cloud costs.

So the question for storage and DevOps teams becomes:

Which PVCs in the cluster are no longer being used by any Pod?

Kubernetes 1.37 introduces the "PersistentVolumeClaimUnusedSinceTime" feature to make this information available directly through the PVC status.

Once enabled, the PVC Protection Controller adds an "Unused" condition to the PVC:

- "Unused=False" → A running or pending Pod references the PVC
- "Unused=True" → No non-terminal Pod references the PVC

This makes it much easier to identify potentially orphaned storage and build cleanup automation around questions like:

👉 "Which PVCs have been unused for 30+ days?"

A small addition to Kubernetes, but a useful improvement for storage management, cloud cost control, and cluster operations.

Reference : <https://lnkd.in/gvYfRSUQ>

#Kubernetes #K8s #AWS #DevOps #Cloud

742 reactions742

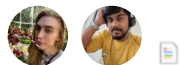
34 comments34 comments

•

54 reposts54 reposts

Comment

Send



<https://www.linkedin.com/in/emmeowzing/>

<https://www.linkedin.com/in/arunfiddler/>

👉 Kubernetes 1.37 quietly introduced something DevOps teams are going to love. In large Kubernetes clusters, it's common for users to create PersistentVolumeClaims (PVCs), use them with Pods, and later delete those Pods without cleaning up the storage. And Kubernetes doesn't automatically delete the PVC when its associated Pod is removed - intentionally, to protect against accidental data loss. Over time, these unused PVCs can accumulate, consuming storage capacity and increasing cloud costs. So the question for storage and DevOps teams becomes: Which PVCs in the cluster are no longer being used by any Pod? Kubernetes 1.37 introduces the "PersistentVolumeClaimUnusedSinceTime" feature to make this information available directly through the PVC status. Once enabled, the PVC Protection Controller adds an "Unused" condition to the PVC: ✅ "Unused=False" → A running or pending Pod references the PVC ✅ "Unused=True" → No non-terminal Pod references the PVC This makes it much easier to identify potentially orphaned storage and build cleanup automation around questions like: 👉 "Which PVCs have been unused for 30+ days?" A small addition to Kubernetes, but a useful improvement for storage management, cloud cost control, and cluster operations. Reference : <https://lnkd.in/gvYfRSUQ> #Kubernetes #K8s #AWS #DevOps #Cloud

Report Last Used Time On a PVC

#Kubernetes

#K8s
#AWS
#DevOps
#Cloud
742 reactions 742

Emma Doyle reposted this

Gjorgji J.

- 3rd+

Orchestrating chaos into seamless deployments 🚀

Oct 3

Your cloud provider loves it when dev clusters run all weekend.

LightsOut, my Kubernetes operator for scaling workloads down during off-hours, has come a long way since my last update. It now hibernates operator-managed resources, scales in ArgoCD sync wave order, and supports FluxCD and HPAs. Still no app code changes, no GitOps rework, and lower costs on autopilot.

How do you keep dev cluster spend in check? I'd love to hear what works for you.

<https://lnkd.in/dC3WYy39>

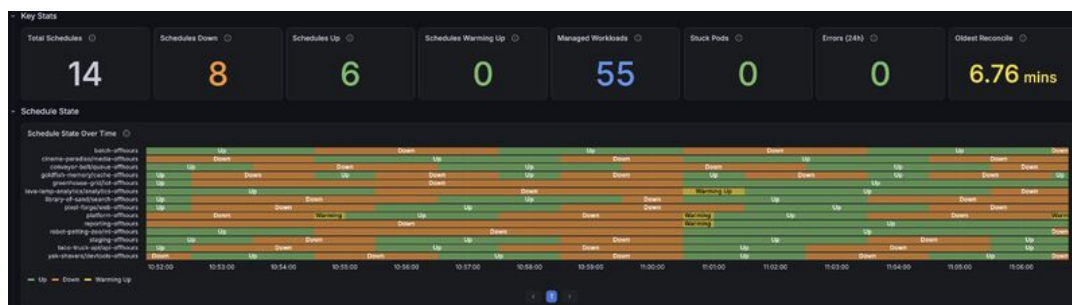
70 reactions70

2 comments2 comments

•

1 repost1 repost

Send



gjorgji-ts/lightsout

LightsOut is a Kubernetes operator that automatically downscales workloads during off-hours to reduce cost and resource usage.



2 Contributors
0 Issues
8 Stars
1 Fork



<https://www.linkedin.com/in/emmeowzing/>

<https://www.linkedin.com/in/gjorgji-ts/>

Your cloud provider loves it when dev clusters run all weekend. LightsOut, my Kubernetes operator for scaling workloads down during off-hours, has come a long way since my last update. It now hibernates operator-managed resources, scales in ArgoCD sync wave order, and supports FluxCD and HPAs. Still no app code changes, no GitOps rework, and lower costs on autopilot. How do you keep dev cluster spend in check? I'd love to hear what works for you. <https://lnkd.in/dC3WYy39>

GitHub - gjorgji-ts/lightsout: LightsOut is a Kubernetes operator that automatically downscales workloads during off-hours to reduce cost and resource usage.

70 reactions 70

Emma Doyle reposted this

Luiza Jarovsky, PhD

- 3rd+

Co-founder of the AI, Tech & Privacy Academy (1,600+ participants), Author of Luiza's Newsletter (100,000+ subscribers), Mother of 3
[View my newsletter](#)

Oct 1

Unpopular opinion: AI companies are pushing society to accept an unacceptable level of risk for a non-essential technology.

833 reactions833

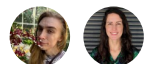
98 comments98 comments

-

45 reposts45 reposts

[Comment](#)

[Send](#)



Luiza's Newsletter

Essays, guides, classes, and workshops on ensuring AI is safe, ethical, and legally compliant. Join 100,000+...

[Subscribe](#)

<https://www.linkedin.com/in/emmeowzing/>

<https://www.linkedin.com/in/luizajarovsky/>

[View my newsletter](#)

Unpopular opinion: AI companies are pushing society to accept an unacceptable level of risk for a non-essential technology.

833 reactions 833

Emma Doyle reposted this

Jered Sutton

- Following

Bit Plumber | Open-source Advocate | AI Engineer

Oct 2

If you are annoyed by the prose your agent is writing, give the following a try in your AGENTS/CLAUDE.md.

5. Prose

- Follow the Google Developer Documentation Style Guide.
- Use ASD-STE100-derived precision rules.
- Apply Zinsser's four principles: clarity, simplicity, brevity, and humanity.

I have found it to reduce verboseness and tighten up precision in the output of my agents. The section number is based on the 4 section CLAUDE.md attributed to Andrej Karpathy.

111 reactions111

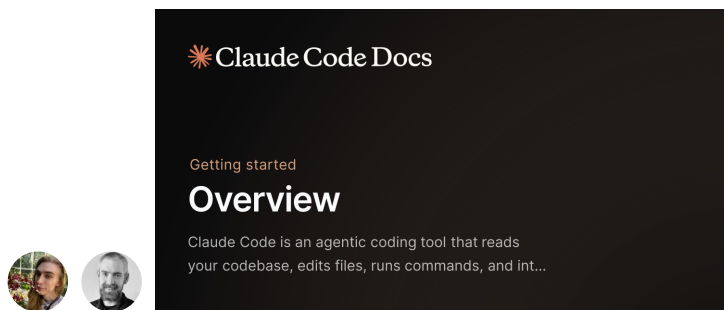
8 comments8 comments

-

6 reposts6 reposts

[Comment](#)

[Send](#)



<https://www.linkedin.com/in/emmeowzing/>

<https://www.linkedin.com/in/jereds/>

If you are annoyed by the prose your agent is writing, give the following a try in your AGENTS/ CLAUDE.md . ## 5. Prose - Follow the Google Developer Documentation Style Guide. - Use ASD-STE100-derived precision rules. - Apply Zinsser's four principles: clarity, simplicity, brevity, and humanity. I have found it to reduce verboseness and tighten up precision in the output of my agents. The section number is based on the 4 section CLAUDE.md attributed to Andrej Karpathy.

CLAUDE.md

111 reactions 111

Emma Doyle reposted this

James Busia

• 3rd+

I code for fun. I build real stuff for real people to really use.

Sep 30

Is it not obvious at this point that the plan is to get as many devs addicted to AI codegen as possible, make them completely reliant on it, make crucial infrastructure impossible to alter without it, then 10x the price?

I've never been more confident in my decision to keep learning and working using my brain.

330 reactions 330

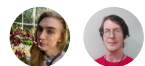
83 comments 83 comments

•

16 reposts 16 reposts

Comment

Send



<https://www.linkedin.com/in/emmeowzing/>

<https://www.linkedin.com/in/james-iden-busia/>

Is it not obvious at this point that the plan is to get as many devs addicted to AI codegen as possible, make them completely reliant on it, make crucial infrastructure impossible to alter without it, then 10x the price? I've never been more confident in my decision to keep learning and working using my brain.

330 reactions 330

Emma Doyle reposted this

Jeff Boggs B.A.Sc. MBA

• 2nd

Creator of Opportunity OS (opportunity-system.com) | Software Architect and Author of AI-Native Software Engineering | 📺 Watch how I build at AI speed

[View my portfolio](#)

Sep 27 • Edited

When an engineer has spent 10 to 20 years mastering manual syntax, compiler quirks, and hand-rolled boilerplate, typing code by hand becomes synonymous with their value.

Having an LLM generate in five seconds what used to take an afternoon of careful keystrokes feels like personal invalidation so the reaction often morphs into pure hostility toward the tools and anyone using them.

But tedious was never the same thing as valuable. The leverage is moving up the stack to systems design and architecture, whether we like it or not.

41 reactions41

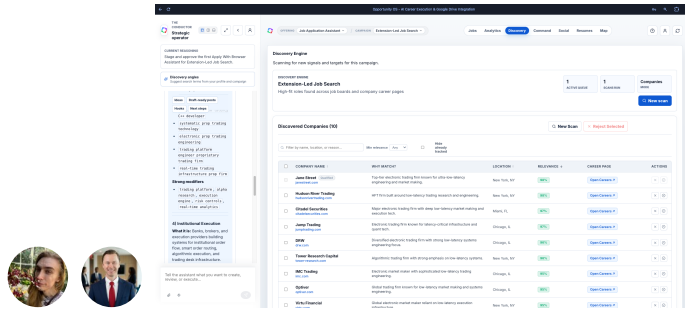
51 comments51 comments

•

2 reposts2 reposts

Comment

Send



<https://www.linkedin.com/in/emmeowzing/>

<https://www.linkedin.com/in/jeffboggs/>

View my portfolio

When an engineer has spent 10 to 20 years mastering manual syntax, compiler quirks, and hand-rolled boilerplate, typing code by hand becomes synonymous with their value. Having an LLM generate in five seconds what used to take an afternoon of careful keystrokes feels like personal invalidation so the reaction often morphs into pure hostility toward the tools and anyone using them. But tedious was never the same thing as valuable. The leverage is moving up the stack to systems design and architecture, whether we like it or not.

41 reactions 41

Emma Doyle

• You

Owner @ Tiger Lily Plants

Sep 26 • Edited

I built a plugin for Helm charts that uses graph-based structural filtering and Python's Hypothesis testing framework to explore input configuration spaces (values.yaml) efficiently, to find the bugs you never thought to test for:

<https://lnkd.in/ektqstTY>

Most Helm testing validates static configurations and doesn't exercise the full space of configurable paths or combinations of values.

That leaves many classes of bugs among structural complexities, unaddressed before release:

- One value change can have downstream effects across multiple rendered resources.
- Individually valid settings can fail when combined.
- configurable fields exist only in templates and aren't documented in values.yaml.
- values paths are effectively dead and never change rendered output.
- Different values within the same type constraints can still produce structurally different behavior.

And all of these cases could pass a static 'helm lint'.

hypothesis-helm attempts to expand automated testing toward the full configuration surface of a chart while filtering redundant cases prior to execution.

The pipeline is roughly:

coalesce values -> pre-filter -> sort -> post-filter -> test -> validate

The tested and rendered manifests that pass hypothesis testing can then be passed through kubesecc or any other tool you already have. I've tried my hand at benchmarking the approach against synthetic charts with varying structures and error distributions, which you can see in the studies/-directory.

I've also tested it against public data, including Bitnami. See the report the tool generates for yourself:

<https://lnkd.in/eQZRNAqJ>

This example includes a lot of diagrams that give us a sense of input value sensitivity in the rendered space, as well as charts' structures.

GitHub - astrivant/hypothesis-helm: Hypothesis testing plugin for Helm

github.com

Nathan Paulhus 🍌 and 1 other reacted

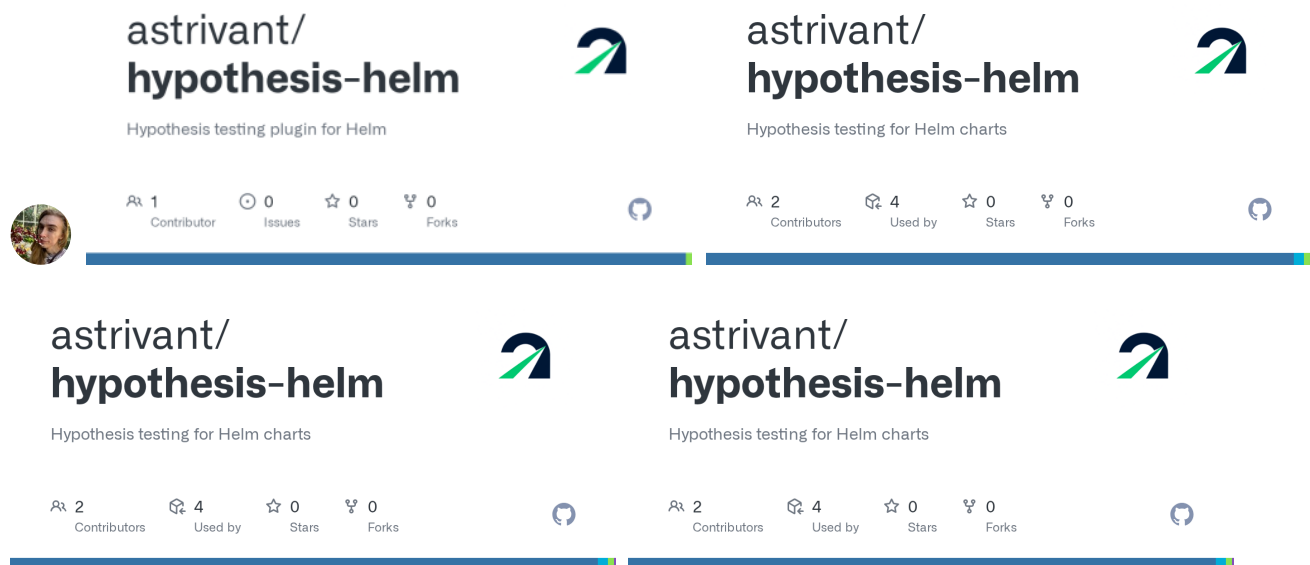
2 comments

Comment

Send

503 impressions

View analytics



<https://www.linkedin.com/in/emmeowzing/>

I built a plugin for Helm charts that uses graph-based structural filtering and Python's Hypothesis testing framework to explore input configuration spaces (values.yaml) efficiently, to find the bugs you never thought to test for: <https://lnkd.in/eKtqstTY> Most Helm testing validates static configurations and doesn't exercise the full space of configurable paths or combinations of values. That leaves many classes of bugs among structural complexities, unaddressed before release: - One value change can have downstream effects across multiple rendered resources. - Individually valid settings can fail when combined. - configurable fields exist only in templates and aren't documented in values.yaml. - values paths are effectively dead and never change rendered output. - Different values within the same type constraints can still produce structurally different behavior. And all of these cases could pass a static 'helm lint'. hypothesis-helm attempts to expand automated testing toward the full configuration surface of a chart while filtering redundant cases prior to execution. The pipeline is roughly: coalesce values -> pre-filter -> sort -> post-filter -> test -> validate The tested and rendered manifests that pass hypothesis testing can then be passed through kubesecc or any other tool you already have. I've tried my hand at benchmarking the approach against synthetic charts with varying structures and error distributions, which you can see in the studies/-directory. I've also tested it against public data, including Bitnami. See the report the tool generates for yourself: <https://lnkd.in/eQZRNAqJ> This example includes a lot of diagrams that give us a sense of input value sensitivity in the rendered space, as well as charts' structures.

GitHub - astrivant/hypothesis-helm: Hypothesis testing for Helm charts

[hypothesis-helm/docs/reports/bitnami.pdf](#) at main · astrivant/hypothesis-helm

GitHub - astrivant/hypothesis-helm: Hypothesis testing plugin for Helm

github.com

Nathan Paulhus 🍌 and 1 other reacted

Emma Doyle reposted this

Aniket Maithani

- Following

Chief Technology Officer @ M37Labs | Building AI and Data Intelligence Platforms

Sep 17

Kafka just quietly fixed one of its oldest limitations. And I'm not sure enough Python teams have noticed yet. With Kafka 4.2, Share Groups (KIP-932) became production-ready.

That sounds like another Kafka feature until you realize what it changes:

You are no longer stuck with "one partition, one consumer" for parallel processing. With Share Groups, multiple workers can pull work from the same partition. Each message gets locked, processed, and acknowledged independently. No awkward rebalance callbacks. No manual offset juggling. No creating hundreds of partitions just because you might need more workers later.

The architecture change is much more interesting.

- Worker count is no longer tied to partition count Previously, if you wanted 200 consumers working in parallel, you probably needed something close to 200 partitions. Now you can scale workers independently.
- One slow message doesn't have to block everything behind it For job-processing workloads, this is huge. A slow API call or poison message doesn't need to hold up an entire partition.
- Retries become much cleaner RELEASE puts the message back for redelivery. `delivery_count()` tells you how many attempts have happened. REJECT lets you stop retrying. A lot of retry plumbing that teams build themselves can move back into Kafka.
- This could remove an entire piece of infrastructure

I've seen architectures running:

Kafka for events RabbitMQ/SQS for jobs custom retry logic another monitoring surface If RabbitMQ or SQS is sitting next to Kafka only because you needed queue-style worker semantics, that second system may no longer be necessary. Of course, there's a trade-off. You lose ordering guarantees inside the Share Group. So if strict ordering matters, classic consumer groups still make sense.

Also worth knowing:

The Java client is GA, while Python's `ShareConsumer` is still Preview. And if you're experimenting on a small dev cluster with fewer than three brokers, check your Share Coordinator state-topic replication settings. Otherwise you may spend an embarrassing amount of time wondering why your consumer is doing absolutely nothing. Ask me how I know. 😊 Kafka is starting to look a lot less like "an event log that people keep forcing into a queue" and a lot more like a system that can comfortably do both. #ApacheKafka #Kafka #Python #DistributedSystems #BackendEngineering #SoftwareArchitecture #EventDrivenArchitecture #DataEngineering

274 reactions274

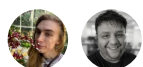
12 comments12 comments

•

21 reposts21 reposts

Comment

Send



<https://www.linkedin.com/in/emmeowzing/>

<https://www.linkedin.com/in/aniketmaithani/>

Kafka just quietly fixed one of its oldest limitations. And I'm not sure enough Python teams have noticed yet. With Kafka 4.2, Share Groups (KIP-932) became production-ready. That sounds like another Kafka feature until you realize what it changes: You are no longer stuck with "one partition, one

consumer" for parallel processing. With Share Groups, multiple workers can pull work from the same partition. Each message gets locked, processed, and acknowledged independently. No awkward rebalance callbacks. No manual offset juggling. No creating hundreds of partitions just because you might need more workers later. The architecture change is much more interesting. 1. Worker count is no longer tied to partition count Previously, if you wanted 200 consumers working in parallel, you probably needed something close to 200 partitions. Now you can scale workers independently. 2. One slow message doesn't have to block everything behind it For job-processing workloads, this is huge. A slow API call or poison message doesn't need to hold up an entire partition. 3. Retries become much cleaner RELEASE puts the message back for redelivery. `delivery_count()` tells you how many attempts have happened. REJECT lets you stop retrying. A lot of retry plumbing that teams build themselves can move back into Kafka. 4. This could remove an entire piece of infrastructure I've seen architectures running: Kafka for events RabbitMQ/SQS for jobs custom retry logic another monitoring surface If RabbitMQ or SQS is sitting next to Kafka only because you needed queue-style worker semantics, that second system may no longer be necessary. Of course, there's a trade-off. You lose ordering guarantees inside the Share Group. So if strict ordering matters, classic consumer groups still make sense. Also worth knowing: The Java client is GA, while Python's `ShareConsumer` is still Preview. And if you're experimenting on a small dev cluster with fewer than three brokers, check your Share Coordinator state-topic replication settings. Otherwise you may spend an embarrassing amount of time wondering why your consumer is doing absolutely nothing. Ask me how I know. 😊 Kafka is starting to look a lot less like "an event log that people keep forcing into a queue" and a lot more like a system that can comfortably do both. #ApacheKafka #Kafka #Python #DistributedSystems #BackendEngineering #SoftwareArchitecture #EventDrivenArchitecture #DataEngineering

#ApacheKafka

#Kafka

#Python

#DistributedSystems

#BackendEngineering

#SoftwareArchitecture

#EventDrivenArchitecture

#DataEngineering

274 reactions 274

Emma Doyle reposted this

Simone Rodigari

• 3rd+

Senior Software Engineer @ Microsoft

[View my blog](#)

Sep 16

Finally polished and published a blog post I started working on a while ago.

I spent some time digging into Cilium's socket LB and how eBPF hooks into the Linux socket lifecycle to load balance Kubernetes Services.

One interesting aspect is that socket LB can select a backend at `connect()` time, before packet construction, avoiding the Linux netfilter conntrack and NAT path for that connection.

It's enabled automatically with Cilium's kube proxy replacement, but can also be used independently.

Writing things down is still one of the best ways I've found to really understand complex topics.

<https://lnkd.in/ebVmCnaP>

How Cilium Uses eBPF Socket Hooks for Kubernetes Service Load BalancingHow Cilium Uses eBPF Socket Hooks for Kubernetes Service Load Balancing

srodi.com

193 reactions193

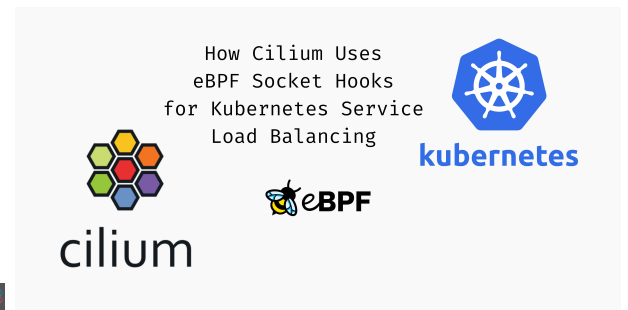
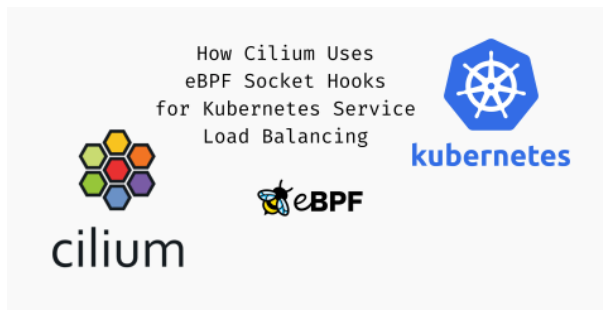
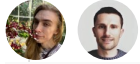
8 comments8 comments

•

15 reposts15 reposts

[Comment](#)

[Send](#)



<https://www.linkedin.com/in/emmeowzing/>

<https://www.linkedin.com/in/srodi/>

View my blog

Finally polished and published a blog post I started working on a while ago. I spent some time digging into Cilium's socket LB and how eBPF hooks into the Linux socket lifecycle to load balance Kubernetes Services. One interesting aspect is that socket LB can select a backend at connect() time, before packet construction, avoiding the Linux netfilter conntrack and NAT path for that connection. It's enabled automatically with Cilium's kube proxy replacement, but can also be used independently. Writing things down is still one of the best ways I've found to really understand complex topics. <https://lnkd.in/ebVmCnaP>

How Cilium Uses eBPF Socket Hooks for Kubernetes Service Load Balancing

How Cilium Uses eBPF Socket Hooks for Kubernetes Service Load Balancing How Cilium Uses eBPF Socket Hooks for Kubernetes Service Load Balancing srodi.com

193 reactions 193

Experience

● Small Business Owner



Tiger Lily Plants LLC. • Self-employed

Nov 2025 - Present • 1 yr

Atlanta, Georgia, United States • On-site

I sell orchids, begonias and carnivorous plants from both my personal collection and wholesalers to exotic plant enthusiasts.

● Lead Platform Engineer



SBE Vision, Inc. • Full-time

Sep 2024 - Sep 2026 • 2 yrs 1 mo

Wakefield, Massachusetts, United States • Remote

- Led platform engineering initiatives in support of SBE Vision's infrastructure across cloud and on-premise environments (prior to- and post-cloud migration).
- Migrated company to cloud infrastructure in GCP / GKE.
- Built and supported ephemeral developer environments to curtail cloud cost.
- Introduced a CI job that traversed deeply-nested Helm dependency trees in a monorepo for MR-based builds based on code change.

- Delivered a fault-tolerant and load tested CI cluster upgrade to improve reliability, scalability, and developer productivity.
- Transitioned secrets management to ESO.
- Delivered a Helm chart / rendered Kubernetes manifest -> Docker compose compiler to reduce time-to-delivery on PoC environments at customer sites, ensuring artifact support / generation and time cost / burden on the team stayed to a minimum.
- Led and mentored engineers by overseeing project execution, providing technical reviews, coordinating platform work, guiding implementation decisions, and supporting team skill set growth.
- Strengthened platform reliability / QoS, automation, and DevOps practices across infrastructure, CI/CD, and developer experience.

● Site Reliability Engineer (Security)



Klaviyo • Full-time

Feb 2024 - Sep 2024 • 8 mos

Boston, Massachusetts, United States • Hybrid

LinkedIn helped me get this job helped me get this job

Technologies:

- Python, Go
- Puppet
- Hashicorp products, such as Terraform, Vault
- AWS (IAM, EC2)

● Founder



PremiScale • Self-employed

Dec 2022 - Sep 2024 • 1 yr 10 mos

Boston, Massachusetts, United States • Hybrid

PremiScale supports projects that scale, accelerate or optimize development and deployment on private or hybrid cloud infrastructure.

Note: this project was on-hold for 2 years as I focused on personal growth in my broader career and starting my small plant business.

● DevOps Engineer



SBE Vision, Inc. • Full-time

Aug 2022 - Feb 2024 • 1 yr 7 mos

Wakefield, Massachusetts, United States • Hybrid

SBE Vision develops an ontology based hub-and-spoke integration platform for interoperability between digital engineering tools.

Responsibilities

- Assisting customers and troubleshooting while deploying SBE Vision's microservice architecture solution to on-premise or cloud environments
- Writing internal technical documentation outlining processes, procedures and standards
- Assisted with disaster recovery in the event of on-premise power outages and hardware or software faults
- Deployed and maintained operations services
- Introduce new security, solutions and customs while practicing DevOps

Projects

- Introduced, maintained and automated an on-premise datacenter based on Rancher / RKE1 to reduce cloud costs and unify cluster access
- Introduced a scalable Kubernetes-based GitLab runners solution
- Centralized and templated Helm charts in a monorepo with supporting GitLab CI trigger framework

- Introduced an IaC solution in Terraform and GitLab CI for managing infrastructure in all public clouds

Technologies

- Helm chart authoring
- Kubernetes (on-premise Rancher RKE1, GKE / EKS / {E,G,A}CR)
- GitLab CI / static runner / kubernetes runner deployment and administration
- Ansible
- Linux KVM virtualization / automation
- Terraform
- Sonatype Nexus
- Git hook authoring
- Rancher deployment and administration
- ZFS
- TLS certificates

● DevOps Engineer



HqO • Full-time

Aug 2021 - Jul 2022 · 1 yr

Boston, Massachusetts, United States · Hybrid

HqO creates workplace experience technology that transforms how people connect with each other and the places they work.

Responsibilities

- Writing technical documentation for engineers outlining procedures, solutions to problems, and onboarding.

Technologies

- CircleCI / Orb authoring
- GitHub Actions
- Argo CD
- Kubernetes / EKS
- Helm chart authoring / Helm chart-testing (ct)
- Spot.io
- Infracost
- Loft.sh / DevSpace
- Terraform / Vault / Hashicorp Cloud
- Artifactory
- Git pre-commit hooks authoring

Education



Monroe Community College

Associate's Degree, Mathematics

2015 - 2016

Projects

resumeme

Oct 2026 - Present



Tiger Lily Plants LLC.



Turn your LinkedIn profile into a polished PDF resume, built and signed with GitHub Actions

Skills

